Informe Técnico: Sistema de Combustible Distribuido

Demian Maturana
Hernan Muñoz
Bruno Ganora

November 7, 2025

Contents

1 Introducción **3**

2 Diseño de la Arquitectura **3**
2.1 Nivel 1: Surtidores . . . 3
2.2 Nivel 2: Distribuidoras . . . 3
2.3 Nivel 3: Empresa . . . 3
2.4 Comunicación entre Capas . . . 3
2.5 Diagrama de la Arquitectura . . . 3

3 Configuración **3**

4 Experimentación **4**
4.1 Actualización de precios de combustibles . . . 4
4.2 Pérdida de conexión . . . 4

5 Análisis de los resultados **4**
5.1 Pro . . . 4
5.2 Debilidades . . . 4

6 Conclusiones **5**

1 Introducción

El presente proyecto aborda la necesidad de una empresa de combustibles de centralizar la gestión de precios y operaciones de sus distribuidoras. La solución implementada consiste en un sistema distribuido que permite a la empresa matriz definir precios de manera remota, mientras que cada distribuidora puede operar de forma autónoma, incluso en caso de pérdida de conexión. El sistema se compone de tres capas principales: la empresa, las distribuidoras y los surtidores de combustible, en donde empresa y distribuidor tienen su propia pagina web, mientras que surtidor es parte de la pagina web de distribuidor.

2 Diseño de la Arquitectura

La arquitectura del sistema se divide en tres niveles, siguiendo el modelo presentado en el punto 2.5.

2.1 Nivel 1: Surtidores

Este nivel está representado por el 'surtidor-simulator', una aplicación en Go que simula el comportamiento de los surtidores de combustible. Cada surtidor se conecta a su distribuidora asignada a través de un socket TCP y envía periódicamente información sobre las ventas realizadas. Los surtidores pueden ser pausados o reanudados de forma remota por la distribuidora.

2.2 Nivel 2: Distribuidoras

El backend de la distribuidora, desarrollado en Go, es el núcleo de este nivel. Actúa como servidor para los surtidores, recibiendo y procesando los datos de las ventas. Cada distribuidora tiene su propia base de datos PostgreSQL para almacenar las transacciones. Además, expone una API REST para la interfaz de usuario ('distribuidor-ui'), que permite monitorear las ventas y el estado de los surtidores.

2.3 Nivel 3: Empresa

La empresa cuenta con una aplicación backend ('empresa') y una interfaz de usuario ('empresa-ui'). El backend está diseñado para comunicarse con las distribuidoras, obtener reportes y gestionar los precios de manera centralizada. La interfaz de usuario permite a la empresa monitorear el estado de las diferentes distribuidoras.

2.4 Comunicación entre Capas

La comunicación entre los surtidores y las distribuidoras se realiza a través de sockets TCP, utilizando un protocolo de mensajería simple basado en texto. La comunicación entre la empresa y las distribuidoras se realiza a través de API REST, lo que facilita la integración y el intercambio de datos en formato JSON.

2.5 Diagrama de la Arquitectura

3 Configuración

Para ejecutar la solución, es necesario contar con Docker y Docker Compose. El archivo 'docker-compose.yml' orquesta el despliegue de todos los servicios, incluyendo las bases de datos. Cada servicio se configura a través de variables de entorno definidas en el mismo archivo, como las credenciales de la base de datos, los puertos de red y los identificadores de las distribuidoras.

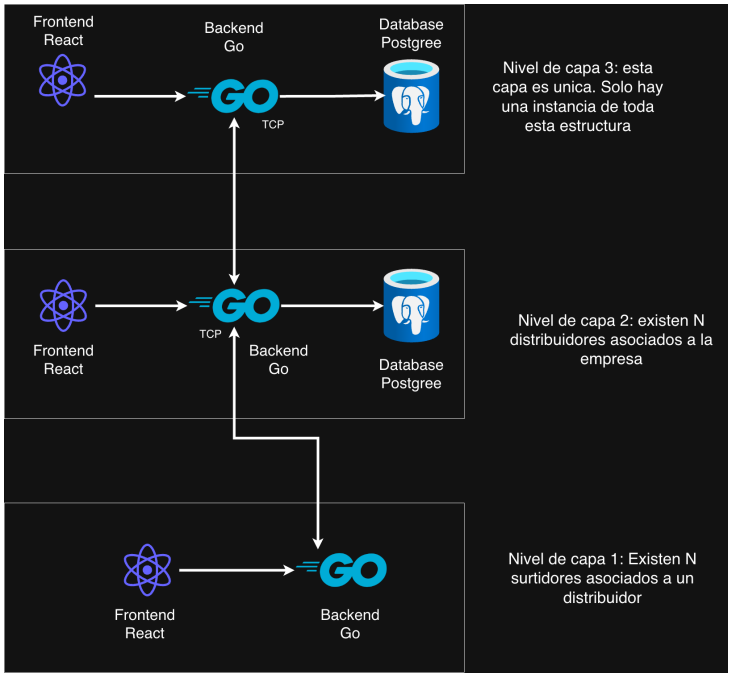


Figure 1: Descripción de la imagen

4 Experimentación

Casos de prueba testeados y que se presentaran en el video.

4.1 Actualización de precios de combustibles

La empresa podría enviar una solicitud de actualización de precios a una distribuidora. Se debería verificar que la distribuidora actualiza su factor de utilidad y que los nuevos precios se aplican a las ventas futuras. Esto se puede probar con surtidores disponibles y ocupados (pausados).

4.2 Pérdida de conexión

Se puede simular una pérdida de conexión entre una distribuidora y la empresa (por ejemplo, deteniendo el contenedor de la empresa). La distribuidora debe seguir operando y almacenando las ventas localmente. Al restablecer la conexión, se debería sincronizar la información.

5 Análisis de los resultados

5.1 Pro

- **Autonomía de las distribuidoras:** La arquitectura permite que cada distribuidora opere de forma independiente, lo que aumenta la resiliencia del sistema.
- **Escalabilidad:** El uso de contenedores facilita el despliegue de nuevas distribuidoras.
- **Flexibilidad tecnológica:** La elección de Go para el backend y React para el frontend demuestra la viabilidad de utilizar diferentes tecnologías en las distintas capas del sistema.

5.2 Debilidades

- **Generacion de reportes:** Aunque el enunciado no lo especifique, se tenia pensado que la empresa y los distribuidores pudieran generar reportes. Por temas de tiempo, no se llego a implementar.
- **Seguridad:** La comunicación entre los componentes no está cifrada, lo que representa un riesgo de seguridad.

6 Conclusiones

El proyecto establece una base sólida para un sistema de gestión de combustible distribuido. Las nuevas tecnologias aprendidas y aplicadas como Go, React y Docker, son adecuadas para este sitema y fue un gran acierto haberlas aprendido.

Se aspiraba mucho en este proyecto por lo que se termino implementando mas de lo solicitado, pero aun asi quedaron bastantes ideas sin implementar por escases de tiempo.

Se espera que se siga con este mismo proyecto en la unidad 3 para poder implementar todo lo pensando y crear un proyecto que parezca lo mas profesional posible.